MCTR1010- Image Processing for Mechatronics

Obstacle Avoidance Vehicle

Final Report

Team 12

Team Member	ID	Tutorial
Abdallah Wael	55-1612	T-2
Sherif Ahmed	55-0828	T-2
Omar Tamer	55-0483	T-1
Mahmoud Hegazy	55-6261	T-4
Marwan Elzahar	55-12704	T-5
Ahmed Yousry	52-8373	T1

Abstract

This project describes the design of an obstacle avoidance system for a mobile robot that uses a Raspberry Pi and Arduino. It employs a webcam to acquire live images that are processed through computer vision algorithms written in Python with the use of the OpenCV library. The core aim of the system is to facilitate object detection and control of robot movement in order to prevent collision and maintain progress.

The image processing framework incorporates color segmentation based on the HSV color model to segment out the target object from the background, and morphological operations to refine the detection process. The object position is computed using the analysis of contours and centroid detection, and the information obtained is used for making control decisions. PD control is used to provide smooth control actions for steering the robot away from obstacles.

The processed data is transmitted via serial communication to an Arduino microcontroller, which controls the DC motor for propulsion and a servo motor for steering. Experimental results demonstrate that the system is capable of real-time obstacle detection and avoidance with stable performance under controlled conditions. The project highlights the integration of image processing, control systems, and embedded hardware in a closed-loop mechatronic system.

Chapter 1
Introduction

Autonomous navigation and obstacle avoidance are fundamental challenges in the field of robotics and mechatronics. With the increasing demand for intelligent systems in applications such as self-driving vehicles, industrial automation, and service robotics, the ability of a system to perceive its environment and react accordingly has become a critical requirement. Vision-based systems, in particular, have gained significant attention due to their ability to provide rich environmental information using relatively low-cost hardware.

This project focuses on the design and implementation of a vision-based obstacle avoidance system using a Raspberry Pi and Arduino platform. The system integrates image processing techniques with embedded control to create a closed-loop robotic system capable of detecting obstacles and autonomously adjusting its motion to avoid collisions. A USB camera is used to capture real-time video input, which is processed on the Raspberry Pi using the OpenCV library in Python.

To ensure robust and reliable obstacle detection, the system employs color-based segmentation in the HSV color space, allowing for effective isolation of a target object under controlled lighting conditions. Morphological operations are applied to enhance the quality of the detected regions by reducing noise and filling gaps. The system then extracts meaningful features from the processed image, specifically the centroid of the detected object, which provides information about its position relative to the robot.

Based on this positional information, a control algorithm is implemented to generate appropriate motion commands. A Proportional-Derivative (PD) controller is used to produce smooth steering responses, enabling the robot to maneuver around obstacles while maintaining forward motion. Additionally, adaptive

speed control is incorporated to improve safety by reducing the robot's velocity when obstacles are detected in close proximity.

The control commands generated by the Raspberry Pi are transmitted to an Arduino microcontroller via serial communication. The Arduino is responsible for actuating the system's hardware components, including a DC motor for propulsion and a servo motor for steering. This integration results in a complete mechatronic system that continuously senses, processes, and reacts to its environment in real time.

The objective of this project is not only to achieve functional obstacle avoidance but also to demonstrate the effectiveness of combining image processing, control theory, and embedded systems in a unified framework. The system is evaluated under various test scenarios to assess its performance in terms of responsiveness, stability, and accuracy. The results highlight both the strengths and limitations of the approach, providing insight into potential areas for future improvement.

Chapter 2

Methodology

2.1. System Overview

The components include the Raspberry Pi as the processor, USB camera as the sensor, and Arduino Uno that controls motor movements. The images captured by the Raspberry Pi are analyzed using OpenCV and then the commands sent through serial communication to the Arduino Uno that controls the motor and steering servo accordingly.

The complete setup can be regarded as a closed-loop control system where the camera is used as a feedback element that helps update robot movement in response to obstacle positions.

2.2. Image Processing Pipeline

2.2.1. Image Acquisition

The system captures real-time images using a USB webcam connected to the Raspberry Pi which is used to detect the obstacles for avoidance.

2.2.2. Color Segmentation

The step that will be taken to detect the obstacle involves the conversion of the captured picture into the HSV format from the RGB color format. It is more reliable for color detection regardless of the lighting condition. The target object (in blue color) will be isolated using a set range of HSV color values in the form of a binary mask.

The pixels within the selected HSV color value will be retained while suppressing all other pixels not within the chosen HSV value range.

2.2.3. Morphological Operations

To improve detection quality and reduce noise, morphological operations are applied to the binary mask. An opening operation (erosion followed by dilation) removes small noise regions, while a closing operation (dilation followed by erosion) fills small gaps within the detected object. These operations enhance the consistency and reliability of the detected object shape.

2.2.4. Contour Detection

Contour extraction is performed on the preprocessed binary image for finding connected components that correspond to possible objects. The largest contour is chosen as the object of interest, with the assumption that it corresponds to the object being detected. There is a minimum area requirement to eliminate false positives.

2.2.5. Centroid Extraction

The centroid of the detected object is computed using image moments. The centroid represents the geometric center of the object and provides a reliable indication of its position within the image frame. The horizontal coordinate of the centroid is used as the primary feature for control decisions.

2.3. Control System

2.3.1. PD Controller

The control approach that will be adopted is the proportional-derivative (PD) technique whose aim will be to provide smooth steering commands from the position of the detected object. The control error will be defined as the difference between the center of the frame and the horizontal position of the object's center of mass.

In PD control, a proportional part gives a response that is directly proportional to the error magnitude while the derivative component provides damping, which means it eliminates the oscillation effect.

2.3.2. Repulsive Behavior

The principle of artificial repulsive fields is considered for improving obstacle avoidance. The more an object approaches the center of the image, the stronger the repulsive force, which causes an increase in the magnitude of the steering correction for avoiding collision.

2.3.3. Speed Control

In addition to steering control, adaptive speed control is implemented. The robot reduces its speed as the obstacle approaches the center of the field of view, thereby minimizing the risk of collision and improving safety during navigation.

2.4. Hardware

2.4.1. List of Components

a. Raspberry Pi
b. Camera
c. Arduino Uno
d. L298N Motor Driver
e. DC Motor
f. Encoder
g. H-Bridge
h. Car case
i. Power Supply
j. Jump wires

2.4.2. Hardware with Connections

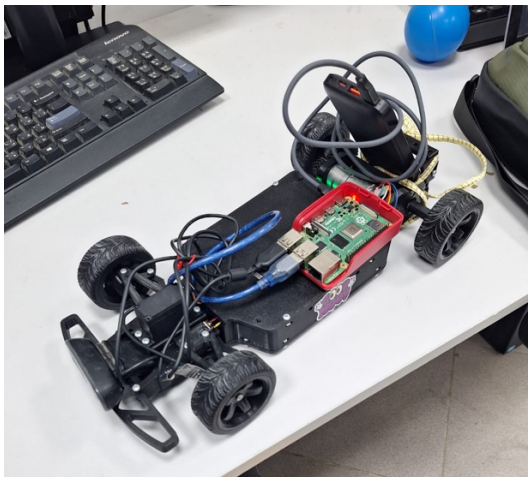

Figure 1. A photo of the 3D printed car having all connections done for the system.

2.5. Diagram and Fabricated circuits

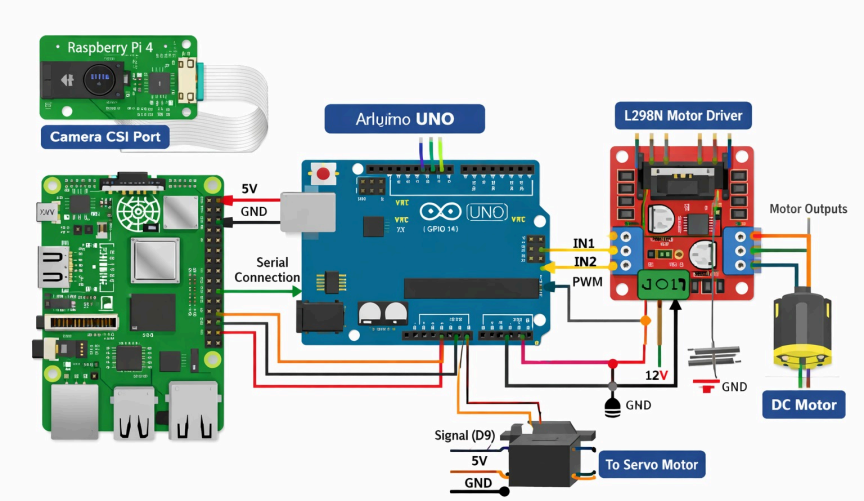

Figure 2. A diagram of the circuit used to run the system.

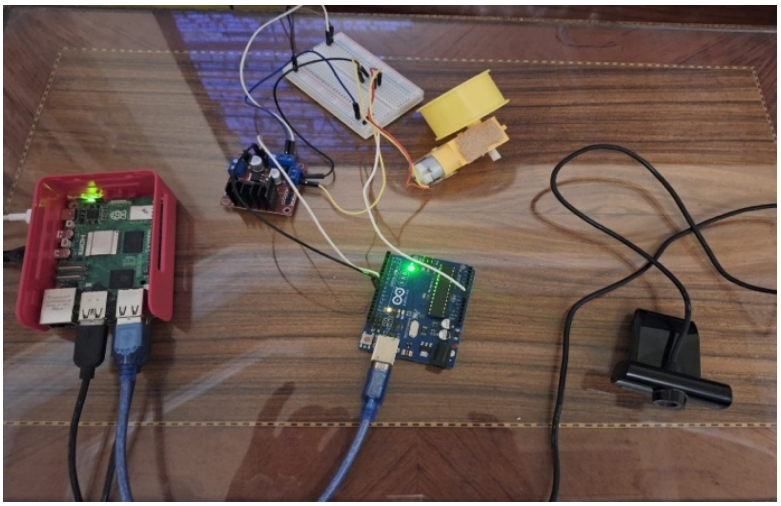

Figure 3. A Photo of the fabricated circuit before being put on the car.

Chapter 3

Results

The experiment was carried out in different settings in order to test the performance of the system and its ability to respond in real time. Different positions, lighting conditions, and distances were taken into account in order to determine the effectiveness of the vision-based control algorithm for robot navigation. As can be seen from the results obtained, the robot is able to effectively detect and avoid obstacles placed at different points inside its camera view range.

When an obstacle is placed in the middle of the camera frame, the system provides a very strong reaction due to the presence of a high control error, resulting in quick avoidance of a possible collision. If the obstacle appears on the left or right sides of the frame, then the robot performs smooth trajectory changes through steering.

The system exhibited stable and consistent performance throughout its operation. The reason for this stability can mainly be explained by temporal filtering, which filters out noise and variations in the position of the object detected, and PD control, which limits oscillation and provides a smooth steering transition. Due to this stability, the robot does not perform sudden movements but navigates in a controlled manner.

Another test involved changing the distance between the robot and the obstacle to see how far it could maintain successful performance. It was noticed that within a reasonable working distance, the system managed to maintain its avoidance capabilities; however, detection sensitivity tended to decrease as the distance increased because of fewer pixels in the image.

Nevertheless, several limitations have been revealed during the operation of the developed system. Performance degradation was observed under poor lighting conditions, affecting the accuracy of color segmentation. Moreover, when obstacles had almost identical colors to that of the background, the process became inefficient leading to inaccurate detection.

Chapter 4

Conclusion

In this project, a complete closed-loop obstacle avoidance system was successfully implemented using image processing and embedded control. The integration of a Raspberry Pi, camera, and Arduino enabled real-time detection and response to environmental changes.

The use of color-based segmentation, centroid extraction, and PD control resulted in smooth and reliable navigation behavior. The system demonstrated effective obstacle avoidance and adaptability under various test scenarios.

Despite its effectiveness, the system has certain limitations, including sensitivity to lighting conditions and reliance on specific object colors. Future improvements could include the integration of machine learning techniques or multi-sensor fusion to enhance robustness and generalization.

Overall, the project achieved its objectives and provided valuable insights into the implementation of vision-based control systems in mechatronics applications.